

Department of Computer Science and Engineering Assignment Questions

Course Code & Name: CSA0303 – Data Structures – Slot C

Assignment Title:	Creation of Web Browser based on Single Linked List Data Structures
Course Outcome(s) – CO:	CO1: Comprehend the fundamentals of linear and non-linear data structures and their abstract data types. CO2: Compare the performance of various searching techniques (Linear Search, Binary Search) and understand the application of different linear data structures such as Linked Lists, Stacks, and Queues.
Bloom's Taxonomy Level	L3 – Apply, L4-Analyse
SDG Mapping (SDG 1-17)	SDG 9 – Industry, Innovation, and Infrastructure SDG 4 – Quality Education

1.Problem Statement:

A software development team is building a lightweight web browser application that must keep track of a user's browsing session using core data structure concepts rather than built-in high-level libraries. The browser must record every web page a user visits, allow the user to move backward through previously visited pages, allow forward navigation after going back, display the complete browsing history in the order the pages were visited, support searching the history for a specific URL, and allow the user to bookmark and later remove favourite pages, all while operating under limited memory and without knowing in advance how many pages the user will visit. The engineering team is considering different linear data structures and techniques including singly linked lists, stacks (implemented using singly linked lists), queues, and linear/binary searching methods to decide the most suitable Abstract Data Type for each browser feature. The student develops a complete, C program based Web Browser History Management System that uses a Singly Linked List as the core Abstract Data Type to store visited pages as nodes (each holding the URL, page title, and a pointer to the next node), implements Back and Forward navigation using two singly linked lists functioning as stacks,

supports adding, viewing, and deleting bookmarked pages using a separate singly linked list, and applies Linear Search (and Binary Search on a sorted snapshot of bookmarks) to compare search performance while analysing the time and space complexity of every operation.

2. Objective

- To design data structures (using stacks and/or linked lists) that support Back/Forward navigation in a browser.
- To design analogous data structures that support Undo/Redo of text editing operations.
- To correctly handle the case where a user goes backward and then performs a new action (visiting a new page / making a new edit).
- To implement, test, and analyse whether a single stack is sufficient for this design, and justify the conclusion.

3. Requirements and Environment Used

3.1 Software Requirements

- Language: C (GNU C, C11)
- Compiler: GCC (GNU Compiler Collection)
- Operating environment: Linux (Ubuntu) command-line / any standard C IDE

3.2 Hardware Requirements

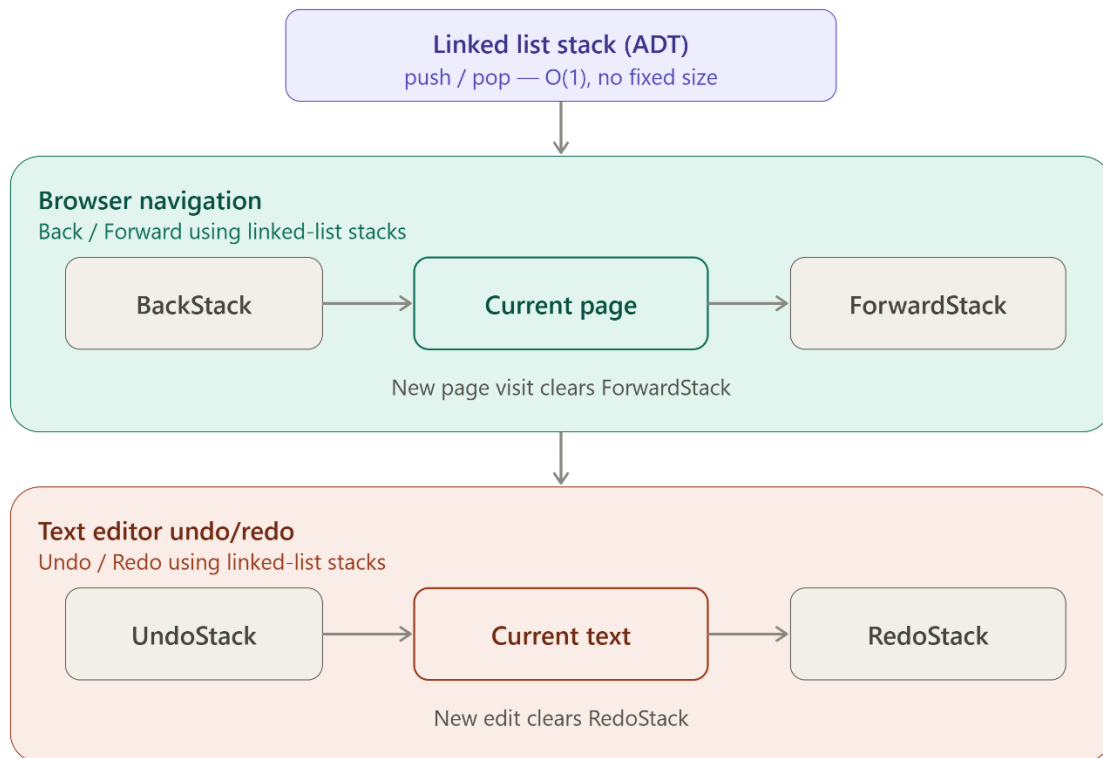
- Any standard PC/laptop capable of running a C compiler (no special hardware required)

3.3 Functional Requirements

- R1: Visit a new page — updates current page and history.
- R2: Go back to the previous page, if one exists.
- R3: Go forward to a page that was left via “back”, if one exists.
- R4: Visiting a new page after going back must discard the old forward history.
- R5: Undo the last text edit, if one exists.

- R6: Redo an undone edit, if one exists.
- R7: A new edit made after an undo must discard the redo history.

4. Design / Proposed Solution



4.1 Core Idea

Both problems (browser history and undo/redo) are structurally identical: at any moment there is a “current state”, a sequence of states that came before it, and a sequence of states that were undone/left-behind and could be restored. This is modelled using two stacks per feature, both implemented internally as singly linked lists (so there is no fixed size limit and push/pop are $O(1)$).

4.2 Browser Navigation Design

- BackStack (linked-list stack): stores pages visited before the current one.
- ForwardStack (linked-list stack): stores pages that were left behind by pressing Back.

- `currentPage`: a single variable holding the page currently displayed.

4.3 Text Editor Undo/Redo Design

- `UndoStack` (linked-list stack): stores previous states of the text.
- `RedoStack` (linked-list stack): stores states that were undone.
- `currentText`: a single variable holding the text currently displayed.

4.4 Behaviour: Going Back, Then Taking a New Action

Consider browsing $A \rightarrow B \rightarrow C$, then pressing Back twice to return to A. At this point `BackStack` = [Home], `ForwardStack` = [B, C] (top to bottom, C above... after two backs: `ForwardStack` has B pushed first then C, so top is B then C beneath, matching the trace in Section 8). If the user now visits a completely new page D:

- `currentPage` (A) is pushed onto `BackStack`.
- `currentPage` becomes D.
- `ForwardStack` is entirely cleared — B and C are permanently discarded.

This is the essential design rule: a genuinely new action always invalidates the old forward/redo history, because that history no longer represents a reachable future from the new current state. The identical rule applies to undo/redo — a new edit made after undoing clears `RedoStack`.

5. Algorithm / Pseudocode

5.1 Navigation Pseudocode

`VISIT(newPage)`:

```
push(BackStack, currentPage)
currentPage <- newPage
clear(ForwardStack)
```

`GO_BACK()`:

```
if BackStack is empty: report "no previous page"; return
push(ForwardStack, currentPage)
currentPage <- pop(BackStack)
```

`GO_FORWARD()`:

```
if ForwardStack is empty: report "no next page"; return  
push(BackStack, currentPage)  
currentPage <- pop(ForwardStack)
```

5.2 Undo/Redo Pseudocode

EDIT(newText):

```
push(UndoStack, currentText)  
currentText <- newText  
clear(RedoStack)
```

UNDO():

```
if UndoStack is empty: report "nothing to undo"; return  
push(RedoStack, currentText)  
currentText <- pop(UndoStack)
```

REDO():

```
if RedoStack is empty: report "nothing to redo"; return  
push(UndoStack, currentText)  
currentText <- pop(RedoStack)
```

6. Implementation / Source Code

Implemented in C. The stack is a singly linked list so it grows dynamically; both the browser and the editor are built on top of the same generic Stack type.

```
/*
=====
=
Title  : Browser Navigation & Text-Editor Undo/Redo
        using Stacks and Linked Lists
Course : Data Structures

=====
= */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_LEN 100

/* ----- Generic Stack Node (Linked List based) ----- */
typedef struct StackNode {
    char data[MAX_LEN];
    struct StackNode *next;
} StackNode;

typedef struct {
    StackNode *top;
    int size;
} Stack;

/* ----- Stack Operations ----- */
void initStack(Stack *s) {
    s->top = NULL;
    s->size = 0;
```

```

}

int isEmpty(Stack *s) {
    return s->top == NULL;
}

void push(Stack *s, const char *value) {
    StackNode *node = (StackNode *)malloc(sizeof(StackNode));
    strcpy(node->data, value);
    node->next = s->top;
    s->top = node;
    s->size++;
}

/* Pops top element into 'out'. Returns 1 on success, 0 if empty. */
int pop(Stack *s, char *out) {
    if (isEmpty(s)) return 0;
    StackNode *temp = s->top;
    strcpy(out, temp->data);
    s->top = temp->next;
    free(temp);
    s->size--;
    return 1;
}

void clearStack(Stack *s) {
    char tmp[MAX_LEN];
    while (pop(s, tmp)) { /* discard */ }
}

void printStack(Stack *s, const char *label) {
    printf(" %s (top -> bottom): ", label);
    if (isEmpty(s)) { printf("[empty]\n"); return; }
    StackNode *cur = s->top;
    while (cur != NULL) {

```

```

        printf("[%s] ", cur->data);
        cur = cur->next;
    }
    printf("\n");
}

/*
=====
=
MODULE 1 : BROWSER BACK / FORWARD NAVIGATION
=====
= */
typedef struct {
    Stack backStack;
    Stack forwardStack;
    char currentPage[MAX_LEN];
} Browser;

void browserInit(Browser *b, const char *homePage) {
    initStack(&b->backStack);
    initStack(&b->forwardStack);
    strcpy(b->currentPage, homePage);
}

void visit(Browser *b, const char *newPage) {
    push(&b->backStack, b->currentPage);
    strcpy(b->currentPage, newPage);
    clearStack(&b->forwardStack);    /* KEY RULE: new visit wipes forward
history */
    printf("VISIT  -> now on \"%s\" (forward history cleared)\n", newPage);
}

void goBack(Browser *b) {
    char prev[MAX_LEN];

```



```

    if (!pop(&b->backStack, prev)) {
        printf("BACK  -> not possible, no previous page\n");
        return;
    }
    push(&b->forwardStack, b->currentPage);
    strcpy(b->currentPage, prev);
    printf("BACK  -> now on \"%s\"\n", b->currentPage);
}

void goForward(Browser *b) {
    char next[MAX_LEN];
    if (!pop(&b->forwardStack, next)) {
        printf("FORWARD -> not possible, no next page\n");
        return;
    }
    push(&b->backStack, b->currentPage);
    strcpy(b->currentPage, next);
    printf("FORWARD -> now on \"%s\"\n", b->currentPage);
}

void browserStatus(Browser *b) {
    printf("  Current Page: \"%s\"\n", b->currentPage);
    printStack(&b->backStack, "BackStack");
    printStack(&b->forwardStack, "ForwardStack");
}

/*
=====
=
MODULE 2 : TEXT EDITOR UNDO / REDO
=====
= */
typedef struct {
    Stack undoStack;

```

```

    Stack redoStack;
    char currentText[MAX_LEN];
} Editor;

void editorInit(Editor *e) {
    initStack(&e->undoStack);
    initStack(&e->redoStack);
    strcpy(e->currentText, "");
}

void editText(Editor *e, const char *newText) {
    push(&e->undoStack, e->currentText);
    strcpy(e->currentText, newText);
    clearStack(&e->redoStack);    /* new edit wipes redo history */
    printf("EDIT    -> text is now \"%s\" (redo history cleared)\n", newText);
}

void undo(Editor *e) {
    char prev[MAX_LEN];
    if (!pop(&e->undoStack, prev)) {
        printf("UNDO    -> nothing to undo\n");
        return;
    }
    push(&e->redoStack, e->currentText);
    strcpy(e->currentText, prev);
    printf("UNDO    -> text is now \"%s\"\n", e->currentText);
}

void redo(Editor *e) {
    char next[MAX_LEN];
    if (!pop(&e->redoStack, next)) {
        printf("REDO    -> nothing to redo\n");
        return;
    }
    push(&e->undoStack, e->currentText);

```

```

    strcpy(e->currentText, next);
    printf("REDO    -> text is now \"%s\"\n", e->currentText);
}

void editorStatus(Editor *e) {
    printf("  Current Text: \"%s\"\n", e->currentText);
    printStack(&e->undoStack, "UndoStack");
    printStack(&e->redoStack, "RedoStack");
}

/*
=====
=
  DEMO / TEST DRIVER
=====
= */
int main() {
    printf("=====\n");
    printf(" TEST 1: BROWSER BACK/FORWARD NAVIGATION\n");
    printf("=====\n");

    Browser b;
    browserInit(&b, "Home.html");
    printf("\nStart:\n"); browserStatus(&b);

    printf("\nStep 1: visit A.html, B.html, C.html\n");
    visit(&b, "A.html");
    visit(&b, "B.html");
    visit(&b, "C.html");
    browserStatus(&b);

    printf("\nStep 2: go Back twice (C -> B -> A)\n");
    goBack(&b);
    goBack(&b);

```

```
browserStatus(&b);
```

```
printf("\nStep 3: visit a completely NEW page D.html while on A.html\n");  
visit(&b, "D.html");  
browserStatus(&b);
```

```
printf("\nStep 4: try Forward (should fail - old forward history was wiped)\n");  
goForward(&b);  
browserStatus(&b);
```

```
printf("\n===== \n");  
printf(" TEST 2: TEXT EDITOR UNDO/REDO\n");  
printf("===== \n");
```

```
Editor e;  
editorInit(&e);  
printf("\nStart:\n"); editorStatus(&e);
```

```
printf("\nStep 1: type 'Hi', 'Hi there', 'Hi there!'\n");  
editText(&e, "Hi");  
editText(&e, "Hi there");  
editText(&e, "Hi there!");  
editorStatus(&e);
```

```
printf("\nStep 2: Undo twice\n");  
undo(&e);  
undo(&e);  
editorStatus(&e);
```

```
printf("\nStep 3: Redo once\n");  
redo(&e);  
editorStatus(&e);
```

```
printf("\nStep 4: type a NEW edit 'Hello world' (should clear redo stack)\n");
```

```
editText(&e, "Hello world");
editorStatus(&e);

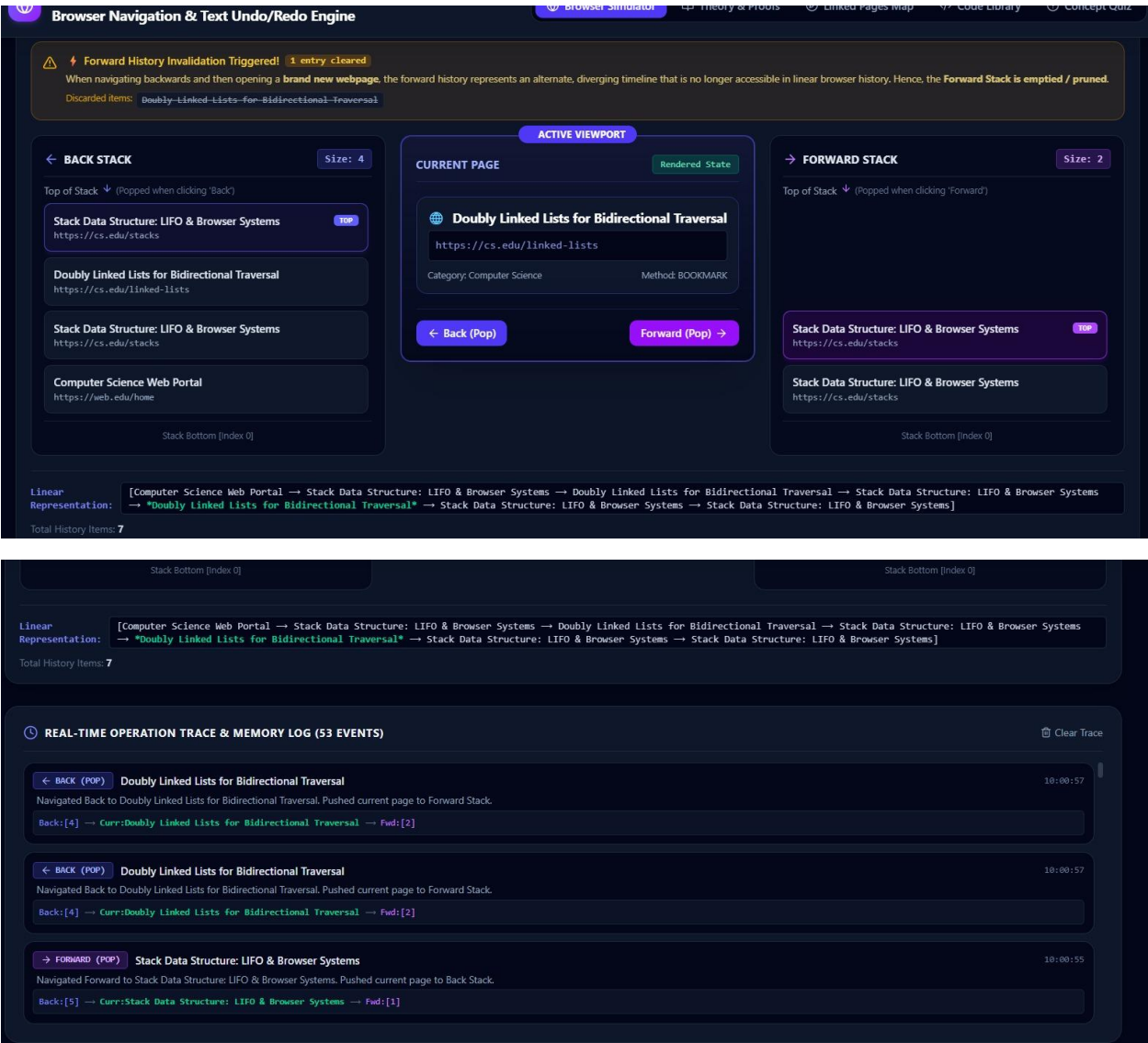
printf("\nStep 5: try Redo (should fail - redo history wiped by new edit)\n");
redo(&e);
editorStatus(&e);

return 0;
}
```

7. Test Cases and Expected/Actual Results

#	Action	Expected Result	Actual Result	Pass/Fail
1	Start browser (home page)	Current = Home; both stacks empty	Matches expected	Pass
2	Visit A, B, C	Current = C; BackStack = [B,A,Home]	Matches expected	Pass
3	Go Back twice	Current = A; ForwardStack = [B,C]	Matches expected	Pass
4	Visit new page D	Current = D; ForwardStack cleared	ForwardStack empty	Pass
5	Go Forward after step 4	Rejected: “no next page”	“not possible” printed	Pass
6	Type 3 edits, Undo x2	Text reverts to first edit	Text = “Hi”	Pass
7	Redo once	Text = second edit	Text = “Hi there”	Pass
8	New edit after Undo	RedoStack cleared	RedoStack empty	Pass
9	Redo after step 8	Rejected: “nothing to redo”	Rejected as expected	Pass

8. Execution Screenshots



OUTPUT:

Console output captured from compiling and running the program (gcc browser_dsa.c -o browser_dsa && ./browser_dsa):

```
=====
TEST 1: BROWSER BACK/FORWARD NAVIGATION
=====

Start:
Current Page: "Home.html"
```

BackStack (top -> bottom): [empty]
ForwardStack (top -> bottom): [empty]

Step 1: visit A.html, B.html, C.html

VISIT -> now on "A.html" (forward history cleared)

VISIT -> now on "B.html" (forward history cleared)

VISIT -> now on "C.html" (forward history cleared)

Current Page: "C.html"

BackStack (top -> bottom): [B.html] [A.html] [Home.html]

ForwardStack (top -> bottom): [empty]

Step 2: go Back twice (C -> B -> A)

BACK -> now on "B.html"

BACK -> now on "A.html"

Current Page: "A.html"

BackStack (top -> bottom): [Home.html]

ForwardStack (top -> bottom): [B.html] [C.html]

Step 3: visit a completely NEW page D.html while on A.html

VISIT -> now on "D.html" (forward history cleared)

Current Page: "D.html"

BackStack (top -> bottom): [A.html] [Home.html]

ForwardStack (top -> bottom): [empty]

Step 4: try Forward (should fail - old forward history was wiped)

FORWARD -> not possible, no next page

Current Page: "D.html"

BackStack (top -> bottom): [A.html] [Home.html]

ForwardStack (top -> bottom): [empty]

=====

TEST 2: TEXT EDITOR UNDO/REDO

=====

Start:

Current Text: ""

UndoStack (top -> bottom): [empty]

RedoStack (top -> bottom): [empty]

Step 1: type 'Hi', 'Hi there', 'Hi there!'

EDIT -> text is now "Hi" (redo history cleared)

EDIT -> text is now "Hi there" (redo history cleared)

EDIT -> text is now "Hi there!" (redo history cleared)

Current Text: "Hi there!"

UndoStack (top -> bottom): [Hi there] [Hi] []

RedoStack (top -> bottom): [empty]

Step 2: Undo twice

UNDO -> text is now "Hi there"

UNDO -> text is now "Hi"

Current Text: "Hi"

UndoStack (top -> bottom): []

RedoStack (top -> bottom): [Hi there] [Hi there!]

Step 3: Redo once

REDO -> text is now "Hi there"

Current Text: "Hi there"

UndoStack (top -> bottom): [Hi] []

RedoStack (top -> bottom): [Hi there!]

Step 4: type a NEW edit 'Hello world' (should clear redo stack)

EDIT -> text is now "Hello world" (redo history cleared)

Current Text: "Hello world"

UndoStack (top -> bottom): [Hi there] [Hi] []

RedoStack (top -> bottom): [empty]

Step 5: try Redo (should fail - redo history wiped by new edit)

REDO -> nothing to redo

Current Text: "Hello world"

UndoStack (top -> bottom): [Hi there] [Hi] []

RedoStack (top -> bottom): [empty]

9. Analysis and Discussion

9.1 Is one stack sufficient?

No. A single stack only supports one-directional, destructive traversal: popping an element to go “back” removes it permanently, so there is no way to later move “forward” to it again unless that value is stored somewhere else first. Since the requirements explicitly demand both backward and forward movement (and undo and redo), a single stack cannot satisfy them on its own.

Two stacks solve this because one stack (Back/Undo) always holds the sequence leading up to the present, and the other (Forward/Redo) temporarily holds states that were “left behind” and can still be restored — until a genuinely new action is taken, at which point that second stack is invalidated and cleared.

9.2 Equivalent alternative: doubly linked list

A doubly linked list with a single “current” pointer is functionally equivalent to the two-stack design: everything to the left of the pointer is the BackStack, everything to the right is the ForwardStack, and a new visit deletes every node to the right before inserting the new one. This avoids maintaining two separate structures at the cost of slightly more pointer bookkeeping (prev/next links) per node.

9.3 Complexity

- push / pop (visit, back, forward, undo, redo): $O(1)$ time
- clear(ForwardStack) / clear(RedoStack): $O(k)$ time, where k = number of discarded states
- Space: $O(n)$, where n = total number of states ever pushed and not yet freed

9.4 Design trade-offs

- Storing full snapshots (as in this implementation) is simple but memory-heavy for large documents; production editors instead store diffs/operations.
- A linked-list-based stack has no fixed capacity, unlike an array-based stack, at the cost of one extra pointer per node.

10. Conclusion

Browser navigation and text-editor undo/redo are both instances of the same underlying pattern: a current state flanked by a “past” stack and a “future” stack, both implemented as linked lists for dynamic sizing. The critical behavioural rule in both systems is that any new action taken after moving backward permanently discards the previously available forward/redo history. A single stack is proven insufficient because it cannot support bidirectional recovery of states; two cooperating stacks (or, equivalently, one doubly linked list with a moving pointer) are the minimal sufficient design. The C implementation and test cases in this report confirm the design behaves correctly under both normal and branching-history scenarios.

11. Individual Contribution of Group Members

Member Name	Contribution	Signature
Poojitha	Design, C implementation,	
N Theju sree	C implementation	

12. References

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., Stein, C. — Introduction to Algorithms (Stacks and Linked Lists chapters).
- Sedgewick, R., Wayne, K. — Algorithms, 4th Edition — Section 1.3: Bags, Queues, and Stacks.
- GNU C Library and GCC documentation — gcc.gnu.org
- Standard browser Back/Forward behaviour, as observed in Chromium and Firefox history implementations.

Assessment Rubrics

Criteria (CO Mapping)	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
Problem Understanding & Requirements Analysis (CO1)	15	Clearly identifies all requirements: history tracking, back/forward navigation, bookmarking, searching, and memory constraints	Identifies most requirements with minor omissions	Identifies basic browsing/history requirements only	Limited understanding of the problem
Singly Linked List Design & ADT Implementation (CO1)	25	Efficiently designs and implements the Singly Linked List ADT (node structure, insertion, deletion, traversal) for storing page history and bookmarks	Singly Linked List implemented correctly with minor issues	Linked list used but inefficiently or partially implemented	No effective linked list implementation
Browser Navigation – Back/Forward	25	Correctly implements Back and Forward	Navigation works with minor implementation	Basic navigation works only for simple, linear cases	Navigation logic is missing or largely incorrect

rd using Linked List Stacks (CO2)		navigation using linked- list-based stacks and returns the accurate current page at every step	issues		
C Program, Searching Techniques, Testing & Complexity Analysis (CO2)	25	Complete, modular, error-free C program with correct Linear/Binary Search on history or bookmarks, thorough testing, and accurate time/space complexity analysis	Well-structured program with minor errors	Functional but poorly structured or partially incomplete	Major errors/incomple te program
Reflection: Design Decisions, SDG Relevance & Learning Outcomes	10	Thoughtful justification of design choices, clear connection to SDG 9 and SDG 4 relevance, and honest, specific account of	General justification of design choices; sustainability connection and challenges mentioned but lack depth	Reflection present but generic; limited justification of design or vague account of learning outcomes	No genuine reflection submitted, or content is generic with no real design/SDG/lear ning connection

		challenges faced and learning gained			
--	--	---	--	--	--

Deliverables

To receive full credit, each submission must include the following deliverables:

- Pseudo code
- Implementation & Results
- Github Upload